

Lekcja

Temat: Transakcja w MySQL

Transakcja w MySQL to grupa operacji SQL wykonywana jako jedna całość.

Transakcja spełnia właściwości **ACID**:

- **A — Atomicity (atomowość)**
Wszystko albo nic. Jeśli jedna operacja się nie uda → cofamy wszystko.
- **C — Consistency (spójność)**
Dane po transakcji muszą być poprawne.
- **I — Isolation (izolacja)**
Transakcje nie przeszkadzają sobie nawzajem.
- **D — Durability (trwałość)**
Po COMMIT dane zostają zapisane na stałe.

Instrukcja

START TRANSACTION

BEGIN

COMMIT

ROLLBACK

SAVEPOINT

ROLLBACK TO SAVEPOINT

RELEASE SAVEPOINT

SET TRANSACTION

Opis

Rozpoczyna transakcję

Skrót do START TRANSACTION

Zatwierdza zmiany

Wycofuje zmiany

Tworzy punkt przywracania

Cofnięcie do punktu

Usunięcie savepoint

Ustawienie parametrów transakcji

Przykład 1:

```
CREATE TABLE konto_bankowe (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  wlasciciel VARCHAR(100),  
  saldo DECIMAL(10,2)  
);  
INSERT INTO konto_bankowe (wlasciciel, saldo)  
VALUES  
('Jan', 5000),  
('Anna', 3000);
```

```
/*  
Od tego momentu zmiany nie są jeszcze zapisane na stałe.  
*/  
START TRANSACTION;  
select * from konto_bankowe;  
  
UPDATE konto_bankowe  
SET saldo = saldo - 1000  
WHERE wlasciciel = 'Jan';  
  
UPDATE konto_bankowe  
SET saldo = saldo + 1000  
WHERE wlasciciel = 'Anna';  
select * from konto_bankowe;  
  
COMMIT; -- Zmiany zostają zapisane na stałe.
```

Przykład 2 błędu i ROLLBACK:

```
START TRANSACTION;  
  
select * from konto_bankowe;  
  
UPDATE konto_bankowe  
SET saldo = saldo - 10000  
WHERE wlasciciel = 'Jan';  
-- Jan ma teraz błędne saldo.  
  
ROLLBACK; -- cofa wszystko od START TRANSACTION  
select * from konto_bankowe;
```

ROLLBACK bez SAVEPOINT:

ROLLBACK:

- cofa WSZYSTKIE zmiany,
- od ostatniego:
 - START TRANSACTION
 - lub BEGIN.

Przykład 3 cofnięcia za pomocą SAVEPOINT:

```
START TRANSACTION;
```

```
UPDATE konto SET saldo = saldo - 100 WHERE id = 1;  
SAVEPOINT s1;
```

```
UPDATE konto SET saldo = saldo + 100 WHERE id = 2;  
ROLLBACK TO SAVEPOINT s1;
```

```
COMMIT;
```

Przykład 4 po COMMIT rollback już nie działa

```
START TRANSACTION;
```

```
UPDATE konto  
SET saldo = 0;  
COMMIT;
```

```
ROLLBACK; -- NIE zadziała
```

Autocommit

```
SET autocommit = 1; -- każda instrukcja jest automatycznie commitowana
```

W MySQL domyślnie włączony jest tryb **autocommit=1**.

Oznacza to, że każda pojedyncza instrukcja **INSERT/UPDATE/DELETE** jest traktowana jako osobna transakcja. **START TRANSACTION** lub **BEGIN** tymczasowo wyłącza autocommit aż do **COMMIT/ROLLBACK**.

Lekcja [Tej lekcji jeszcze nie było, dopracować]

Temat: Bezpieczeństwo w MySQL – kluczowe obszary

Transparent Data Encryption (TDE)

Transparent Data Encryption (TDE) to mechanizm szyfrowania danych „w spoczynku” (data at rest), który zapewnia ochronę danych zapisanych na dysku. Dane są automatycznie szyfrowane podczas zapisu i odszyfrowywane podczas odczytu, bez konieczności modyfikacji zapytań SQL przez użytkownika.

Z perspektywy użytkownika i aplikacji proces szyfrowania jest całkowicie transparentny.

Zakres szyfrowania TDE:

- pliki tabel InnoDB,
- logi redo/undo,
- pliki tymczasowe,
- backupy (w zależności od konfiguracji).

Ochrona danych przed nieautoryzowanym dostępem na poziomie systemu plików (np. kradzież dysku, kopii zapasowej lub dostęp administracyjny do plików bazy).

Ograniczenia środowiska XAMPP (MariaDB 10.4)

W środowisku MariaDB działającym w XAMPP:

```
SELECT VERSION();  
-- 10.4.32-MariaDB
```

występują ograniczenia w zakresie monitorowania i zarządzania szyfrowaniem na poziomie tablespace.

Próba odczytu metadanych:

```
SELECT *  
FROM information_schema.INNODB_TABLESPACES;
```

zwraca błąd:

```
#1109 - Unknown table 'innodb_tablespaces' in information_schema
```

Oznacza to brak dostępności tej struktury systemowej w danej wersji środowiska, co uniemożliwia pełne monitorowanie szyfrowania na poziomie tablespace.

Key Management i ograniczenia TDE w XAMPP

W środowisku XAMPP dostępna jest składnia:

```
ENCRYPTION = 'Y'
```

jednak jej pełne wykorzystanie wymaga systemu zarządzania kluczami (key management), który nie jest dostępny w domyślnej instalacji MariaDB XAMPP.

Weryfikacja dostępnych pluginów:

```
SHOW PLUGINS;
```

Brak następujących komponentów oznacza brak systemu zarządzania kluczami:

- `file_key_management`
- `keyring_file`
- `innodb_key_management`

W konsekwencji mechanizm TDE nie może działać w pełnym zakresie, mimo dostępności składni SQL.

Zastosowane podejście – szyfrowanie kolumn (AES)

W związku z ograniczeniami środowiska zastosowano szyfrowanie na poziomie aplikacyjnym przy użyciu funkcji:

- `AES_ENCRYPT()`
- `AES_DECRYPT()`

Przykład 1

```
CREATE TABLE users (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    pesel BLOB );
```

```
INSERT INTO users (pesel) VALUES  
(AES_ENCRYPT('12345678901', 'secret_key'));
```

```
SELECT AES_DECRYPT(pesel, 'secret_key')  
FROM users;
```

```
SELECT CAST(AES_DECRYPT(pesel, 'secret_key') AS CHAR) FROM users;
```

Przykład 2

```
CREATE TABLE customers (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100),  
    pesel VARBINARY(255),  
    credit_card VARBINARY(255));
```

```
INSERT INTO customers (name, pesel, credit_card) VALUES  
( 'Jan Kowalski', AES_ENCRYPT('12345678901', 'secret_key'), AES_ENCRYPT('1111-2222-  
3333-4444', 'secret_key') );
```

```
SELECT name,  
       CAST(AES_DECRYPT(pesel, 'secret_key') AS CHAR) AS pesel,  
       CAST(AES_DECRYPT(credit_card, 'secret_key') AS CHAR) AS credit_card  
FROM customers;
```

W środowisku XAMPP opartym o MariaDB 10.4 mechanizmy TDE są ograniczone ze względu na brak systemu zarządzania kluczami oraz brak pełnej obsługi metadanych `tablespace`. W związku z tym w analizie bezpieczeństwa zastosowano szyfrowanie na poziomie kolumn przy użyciu funkcji `AES_ENCRYPT()` i `AES_DECRYPT()`, które zapewniają ochronę danych w spoczynku na poziomie logicznym.

Lekcja

Temat: SQL Injection

SQL Injection (SQLi) to rodzaj ataku na aplikacje webowe, w którym ktoś **wstrzykuje własny kod SQL do zapytania bazy danych**, żeby zmienić jego działanie.

Przykład zastosowania:

```
CREATE TABLE uzytkownik (  
id INT PRIMARY KEY AUTO_INCREMENT,  
login VARCHAR(50),  
haslo VARCHAR(50)  
);
```

```
INSERT INTO uzytkownik (login, haslo) VALUES ('admin', '1234');
```

```
INSERT INTO uzytkownik (login, haslo) VALUES ('jan', 'abcd');
```

```
INSERT INTO uzytkownik (login, haslo) VALUES ('anna', 'pass123');
```

Poprawne dane logowania. Aplikacja buduje zapytanie:

```
SELECT * FROM users WHERE login = 'admin' AND password = '1234';
```

SQL Injection (atak)

Atakujący wpisuje:

- login: admin' --
- hasło: cokolwiek (np. xyz)

Aplikacja wykonuje

Wstawia to do zapytania:

```
SELECT * FROM users WHERE login = 'admin' -- ' AND password = 'xyz';
```

Lekcja

Temat: SQL Injection – zabezpieczenie kodu w PHP

1. Prepared Statements (ochrona przed SQL Injection)

```
$stmt = $mysqli->prepare("SELECT username, password FROM users WHERE username = ?");  
$stmt->bind_param("s", $username);  
$stmt->execute();
```

To najważniejsze zabezpieczenie przeciwko atakom SQL Injection.

Zamiast wklejać dane użytkownika bezpośrednio do zapytania SQL (co pozwalało na klasyczne ' OR '1'='1'), kod używa **prepared statement** (parametryzowanego zapytania).

- ? to placeholder.
- bind_param("s", \$username) mówi bazie: „traktuj \$username jako zwykły string, nie jako kod SQL”.
- Serwer bazy danych rozdziela dane od kodu SQL, więc nawet jeśli ktoś wpisze ' OR '1'='1' – zostanie to potraktowane jako nazwa użytkownika, a nie warunek logiczny.

2. password_hash / password_verify (bezpieczne przechowywanie i weryfikacja haseł)

```
if ($user && password_verify($password, $user['password']))
```

- Hasła **nigdy nie są przechowywane w postaci jawnej**.
- W bazie znajdują się hashe wygenerowane przez password_hash(\$pass, PASSWORD_BCRYPT) (lub nowsze algorytmy).
- password_verify() porównuje podane hasło z hashem w bezpieczny sposób (timing-safe comparison).

Dlaczego to ważne?

- Nawet jeśli atakujący wykradnie całą bazę danych, nie dostanie haseł w formie czytelnej.
- Bcrypt automatycznie dodaje salt i jest wolny (co utrudnia brute-force na GPU).

3. Walidacja loginu (regex)

```
if (!preg_match('/^[a-zA-Z0-9_]{3,20}$/', $username))
```

Bardzo restrykcyjna biała lista (whitelist) dopuszczalnych znaków:

- Tylko litery (a-z, A-Z), cyfry (0-9) i podkreślenie _
- Długość od 3 do 20 znaków

Zalety:

- Uniemożliwia wprowadzenie niebezpiecznych znaków (np. ' , " , < , > , ; , -- itp.).
- Zmniejsza powierzchnię ataku (mniej wektorów do SQLi, XSS, Path Traversal itp.).
- Chroni przed wieloma rodzajami ataków na poziomie aplikacji.

4. CSRF Token (ochrona przed Cross-Site Request Forgery)

```
if (empty($_SESSION['csrf_token'])) {  
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));  
}
```

// Sprawdzenie

```
if (!isset($_POST['csrf_token']) || $_POST['csrf_token'] !== $_SESSION['csrf_token']) {  
    die("Nieprawidłowy token CSRF.");  
}
```

Atak CSRF polega na tym, że zalogowany użytkownik niechcący (np. przez złośliwy link lub obrazek) wykonuje akcję na stronie (w tym przypadku logowanie lub inną akcję POST)

Rozwiązanie:

- Generowany jest losowy, kryptograficznie silny token (32 bajty → 64 znaki hex).
- Token jest wysyłany jako ukryte pole w formularzu.
- Serwer sprawdza, czy token z formularza zgadza się z tym zapisanym w sesji.

Dodatkowo token jest generowany tylko raz na sesję (dobra praktyka).

5. Brute Force Protection (ochrona przed atakami siłowymi)

```
if (!isset($_SESSION['login_attempts'])) $_SESSION['login_attempts'] = 0;
```

```
$_SESSION['login_attempts']++;
```



```
if ($_SESSION['login_attempts'] >= 5) {  
    $_SESSION['blocked_until'] = time() + 300; // 5 minut  
}
```

- Licznik nieudanych prób logowania przechowywany w sesji.
- Po 5 nieudanych próbach konto (lub raczej sesja/IP) jest blokowane na 5 minut.
- Po poprawnym zalogowaniu licznik jest zerowany.

Zalety:

- Drastycznie spowalnia atak brute-force.
- Blokada czasowa (rate limiting) jest jedną z najskuteczniejszych metod.

Uwaga: W produkcji lepszym rozwiązaniem jest blokada na poziomie IP + account lockout + CAPTCHA po kilku próbach.

6. Bezpieczne komunikaty błędów

```
if ($mysqli->connect_error) {  
    die("Błąd połączenia z bazą danych."); // nie pokazuje szczegółów  
}
```

Zamiast pokazywać:

Warning: mysqli::prepare(): (HY000/1146) Table 'CTF.users' doesn't exist
...kod zwraca generyczny komunikat.

Szczegółowe błędy to skarbnica informacji dla atakującego (nazwa bazy, wersja MySQL, struktura tabel, ścieżki itp.).

7. htmlspecialchars (ochrona przed XSS)

```
$message = "... Witaj: <strong>" . htmlspecialchars($user['username']) . "</strong>";  
htmlspecialchars() zamienia znaki specjalne:
```

- < → <
- > → >
- & → &
- " → "
- ' → '

Dzięki temu nawet jeśli login zawiera <script>alert(1)</script>, zostanie wyświetlony jako zwykły tekst, a nie wykonany kod JavaScript.

Lekcja

Temat: Ćwiczenia praktyczne tworzenia bazy danych, tabel oraz wypełniania danymi

Zadanie 1 – Biblioteka

Utwórz bazę danych **biblioteka**.

Następnie utwórz tabelę **ksiazki** zawierającą następujące kolumny:

Kolumna	Typ danych
id	INT
tytul	VARCHAR(100)
autor	VARCHAR(100)
rok_wydania	YEAR
liczba_stron	INT
cena	DECIMAL(8,2)
dostepna	BOOLEAN

Polecenia

1. Utwórz bazę danych.
2. Utwórz tabelę książki.
3. Ustaw kolumnę id jako klucz główny z automatycznym numerowaniem.
4. Dodaj samodzielnie minimum 10 książek.
5. Wyświetl wszystkie rekordy.
6. Wyświetl tylko książki droższe niż 50 zł.
7. Posortuj książki według roku wydania malejąco.

Zadanie 2 – Sklep internetowy

Utwórz bazę danych sklep.

Następnie utwórz tabelę **produkty** zawierającą następujące kolumny:

Kolumna	Typ danych
id	INT
nazwa	VARCHAR(100)
kategoria	VARCHAR(100)
cena	DECIMAL(10,2)
stan_magazynowy	INT
producent	VARCHAR(100)
data_dodania	DATE

Polecenia

1. Utwórz tabelę.
2. Wprowadź samodzielnie co najmniej 15 produktów.
3. Wyświetl produkty z kategorii „Elektronika”.
4. Wyświetl produkty z ceną pomiędzy 100 a 500.
5. Posortuj produkty według ceny rosnąco.
6. Policz liczbę produktów każdego producenta.

Lekcja

Temat: Ćwiczenia praktyczne z metodami i funkcjami tekstowymi w MySQL

```
CREATE TABLE users (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    first_name VARCHAR(20),  
    last_name VARCHAR(20),  
    email VARCHAR(50),  
    city VARCHAR(20),  
    description VARCHAR(10)  
);
```

```
INSERT INTO users(first_name, last_name, email, city, description) VALUES  
('Jan', 'Kowalski', 'jan@gmail.com', 'Warszawa', 'Programista'),  
('Anna', 'Nowak', 'anna@gmail.com', 'Kraków', 'Tester'),  
('Piotr', 'Wiśniewski', NULL, 'Gdańsk', NULL),  
('Maria', NULL, 'maria@gmail.com', NULL, 'DevOps');
```

1. CONCAT()

Łączy kilka tekstów w jeden ciąg znaków.

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM users;
```

Zachowanie dla NULL

Jeżeli choć jeden argument jest NULL, cały wynik jest NULL.

```
SELECT CONCAT('Jan', NULL);
```

Wskazówka: obejście

```
SELECT CONCAT(first_name, ' ', IFNULL(last_name, 'BRAK')) FROM users;
```

2. CONCAT_WS()

WS = With Separator.

Łączy teksty używając wskazanego separatora i pomija wartości NULL.

```
SELECT CONCAT_WS('-', first_name, last_name) FROM users;
```

Ciekawostka

NULL jest pomijany.

```
SELECT CONCAT_WS('-', 'A', NULL, 'B');
```

3. LENGTH()

Zwraca liczbę bajtów zajmowanych przez tekst.

```
SELECT LENGTH('ABC');
```

Polskie znaki:

```
SELECT LENGTH('ą');
```

W UTF-8: 2 lub nawet 4 bajty zależnie od kodowania.

4. CHAR_LENGTH()

Zwraca liczbę znaków w tekście niezależnie od kodowania.

```
SELECT CHAR_LENGTH('ą');
```

5. UPPER() / LOWER()

Zamienia wszystkie litery na wielkie/ Zamienia wszystkie litery na małe.

```
SELECT UPPER(first_name) FROM users;
```

6. SUBSTRING()

Wycina fragment tekstu od wskazanej pozycji.

SUBSTRING(tekst, start, długość)

- start > 0 → liczenie od początku
- start < 0 → liczenie od końca
- długość opcjonalna

```
SELECT SUBSTRING('1234567890abcdefghij', 1, 3);
```

```
SELECT SUBSTRING('1234567890abcdefghij', 4);
```

```
SELECT SUBSTRING('1234567890abcdefghij', -7, 4);
```

```
SELECT SUBSTRING('1234567890abcdefghij', 4, -2);
```

```
SELECT SUBSTRING('1234567890abcdefghij', NULL, 3)
```

```
SELECT SUBSTRING(NULL, 1, 2);
```

7. LEFT() i RIGHT()

Pobiera określoną liczbę znaków od początku tekstu. / Pobiera określoną liczbę znaków od końca tekstu.

```
SELECT LEFT('1234567890',2);
```

```
SELECT LEFT('1234567890',-2);
```

```
SELECT RIGHT('1234567890',2);
```

```
SELECT RIGHT('1234567890',-2);
```

8. TRIM()

Usuwa spacje z początku i końca tekstu.

```
SELECT TRIM(' Jan ');
```

LTRIM()

Usuwa spacje tylko z początku tekstu.

```
SELECT LTRIM(' Jan');
```

RTRIM()

Usuwa spacje tylko z końca tekstu.

```
SELECT RTRIM('Jan ');
```

9. REPLACE()

Zamienia wskazany fragment tekstu na inny.

```
SELECT REPLACE(email,'gmail','wp') FROM users;
```

10. REVERSE()

Odwraca kolejność znaków w tekście.

```
SELECT REVERSE('MYSQL');
```

11. LOCATE()

Zwraca pozycję pierwszego wystąpienia szukanego ciągu.

```
SELECT LOCATE('@', email) FROM users;
```

12. INSTR()

Wyszukuje ciąg znaków i zwraca jego pozycję w tekście.

```
SELECT INSTR(email,'@') FROM users;
```

13. POSITION()

Standardowa SQL-owa wersja funkcji wyszukiującej pozycję tekstu.

```
SELECT POSITION('@' IN email) FROM users;
```

14. LPAD()

Dopełnia tekst z lewej strony wskazanymi znakami do określonej długości.

```
SELECT LPAD('123',5,'0');
```

15. RPAD()

Dopełnia tekst z prawej strony wskazanymi znakami do określonej długości.

```
SELECT RPAD('ABC',6,'*');
```

16. REPEAT()

Powtarza tekst określoną liczbę razy.

```
SELECT REPEAT('*',5);
```

17. SPACE()

Tworzy ciąg składający się z podanej liczby spacji.

```
SELECT CONCAT('A',SPACE(5),'B');
```

18. ASCII()

Zwraca kod ASCII pierwszego znaku tekstu.

```
SELECT ASCII('A');
```

19. CHAR()

Zamienia kod ASCII na odpowiadający mu znak.

```
SELECT CHAR(65);
```

20. FORMAT()

Formatuje liczbę z określoną liczbą miejsc po przecinku oraz separatorami tysięcy.

```
SELECT FORMAT(1234567.89,2);
```

```
SELECT FORMAT(1234567.89754,2);
```

21. ELT()

Zwraca numer pozycji szukanej wartości na liście argumentów.

Pobiera element z listy.

```
SELECT ELT(2,'Java','SQL','Angular');
```

```
SELECT ELT(3,'Java','SQL','Angular');
```

```
SELECT ELT(63,'Java','SQL','Angular');
```

ZADANIA

```
CREATE TABLE users (  
  id INT PRIMARY KEY,  
  first_name VARCHAR(50),  
  last_name VARCHAR(50),  
  email VARCHAR(100),  
  city VARCHAR(50),
```

```
salary DECIMAL(10,2)
);
```

```
INSERT INTO users (id, first_name, last_name, email, city, salary) VALUES
(1, 'Jan', 'Kowalski', 'jan.kowalski@gmail.com', 'Warszawa', 5500.50),
(2, 'Anna', 'Nowak', 'anna.nowak@wp.pl', 'Kraków', 7200.00),
(3, 'Piotr', 'Wiśniewski', 'piotr.wisniewski@gmail.com', 'Gdańsk', 8100.75),
(4, 'Krzysztof', 'Mazur', 'krzysztof.mazur@onet.pl', 'Poznań', 6300.20),
(5, 'Maria', 'Zielińska', 'maria.zielinska@gmail.com', 'Wrocław', 9200.00),
(6, 'Tomasz', NULL, 'tomasz@wp.pl', 'Łódź', 4700.00),
(7, 'Łukasz', 'Kaczmarek', 'lukasz.kaczmarek@gmail.com', 'Lublin', 6800.90),
(8, 'Agnieszka', NULL, 'agnieszka@onet.pl', 'Katowice', 7900.50),
(9, 'Michał', 'Wójcik', 'michal.wojcik@gmail.com', 'Szczecin', 8400.00),
(10, 'Joanna', 'Kamińska', 'joanna.kaminska@wp.pl', 'Bydgoszcz', 6100.30);
```

Zadanie 1 – Pełne dane użytkownika

Wyświetl dane w formacie:

Jan Kowalski - jan.kowalski@gmail.com

Wymagania zastosuj funkcje:

- użyj CONCAT()
- alias kolumny: user_info

Zadanie 2 – Bezpieczne łączenie danych

Niektórzy użytkownicy mają NULL w kolumnie last_name.

Wyświetl:

Jan BRAK

zamiast NULL.

Wymagania zastosuj funkcje:

- CONCAT()
- IFNULL()

Zadanie 3 – Separator między danymi

Wyświetl:

Jan | Kowalski | Warszawa

Wymagania zastosuj funkcje:

- CONCAT_WS()

Zadanie 4 – Wielkie litery

Wyświetl wszystkie imiona zapisane wielkimi literami.

Przykład:

JAN

ANNA

PIOTR

Zadanie 5 – Login z e-maila

Dla adresu:

jan.kowalski@gmail.com

wyświetl tylko część przed znakiem @:

jan.kowalski

Wymagania zastosuj funkcje:

- SUBSTRING()
- LOCATE()

Zadanie 6 – Domena e-mail

Dla adresu:

jan.kowalski@gmail.com

wyświetl:

gmail.com

Wymagania zastosuj funkcje:

- SUBSTRING()
- LOCATE()

Zadanie 7 – Inicjały użytkownika

Dla:

Jan Kowalski

wyświetl:

J.K.

Wymagania zastosuj funkcje:

- LEFT()
- CONCAT()

Zadanie 8 – Ukrywanie części e-maila

Dla:

jan.kowalski@gmail.com

wyświetl:

jan*****

Wymagania zastosuj funkcje:

- LEFT()
- REPEAT()
- CONCAT()

Zadanie 9 – Liczba znaków w imieniu

Wyświetl:

- imię
- liczbę znaków w imieniu

Przykład:

Jan 3

Krzysztof 9

Zadanie 10 – Zamiana domeny

Zamień wszystkie adresy:

@gmail.com

na:

@wp.pl

Zadanie 11 – Odwrócone imię

Dla:

Jan

wyświetl:

naJ

Zadanie 12 – Pozycja znaku @

Wyświetl:

- email
- pozycję znaku @

Wymagania zastosuj funkcje:

- LOCATE()

Zadanie 13 – To samo trzema sposobami

Znajdź pozycję znaku @ używając:

- LOCATE()
- INSTR()
- POSITION()

Porównaj wyniki.

Zadanie 14 – Numer klienta

Wyświetl identyfikator użytkownika jako 6-cyfrowy numer.

Przykład:

1 -> 000001

25 -> 000025

123 -> 000123

Wymagania zastosuj funkcje:

- LPAD()

Zadanie 15 – Wyrównanie kodów

Wyświetl imię dopełnione gwiazdkami do długości 10 znaków.

Przykład:

Jan*****

Anna*****

Wymagania zastosuj funkcje:

- RPAD()

Zadanie 16 – Ozdobna wizytówka

Wyświetl:

***** Jan Kowalski *****

Wymagania:

- REPEAT()
- CONCAT()

Zadanie 17 – Spacje między danymi

Wyświetl:

Jan Kowalski

(5 spacji między wartościami)

Wymagania zastosuj funkcje:

- SPACE()

Zadanie 18 – Kody ASCII

Wyświetl:

- pierwszą literę imienia
- jej kod ASCII

Przykład:

J 74

A 65

Wymagania:

- ASCII()
- LEFT()

Zadanie 19 – Z kodu ASCII do znaku

Wyświetl litery:

A

B

C

korzystając wyłącznie z funkcji CHAR().

Zadanie 20 – Formatowanie wynagrodzenia

Wyświetl:

1234567.89 -> 1,234,567.89

Wymagania:

- FORMAT()

Zadanie 21 – Poziom znajomości SQL

Dla wartości:

1

2

3

wyświetl:

Początkujący
Średniozaawansowany
Zaawansowany
Wymagania:

- ELT()

Zadanie kompleksowe

Wyświetl raport użytkowników w formacie:

ID: 000123 | JAN KOWALSKI | login: jan.kowalski | domena: gmail.com | długość imienia: 3